

Drawing

`std.draw` is a canvas of pixels, a handful of shapes and a PNG writer. There is no graphics card involved and nothing to install: you fill a buffer and save it.

A canvas and some shapes

```
use std.io
use std.draw

fn main() {
    let canvas = draw.canvas(320, 200)
    draw.clear(canvas, draw.rgb(11, 11, 13))

    draw.rect(canvas, 24, 24, 130, 56, draw.rgb(255, 161, 54))
    draw.text(canvas, 36, 44, "SPRFST", draw.rgb(11, 11, 13))
    draw.circle(canvas, 250, 120, 40, draw.rgb(120, 200, 255))
    draw.line(canvas, 0, 199, 319, 0, draw.rgb(48, 44, 40))

    if draw.save_png(canvas, "shapes.png") {
        io.say("wrote shapes.png at {draw.size(canvas)}")
    } else {
        io.say("the file could not be written")
    }
}
```

Colours are made with `draw.rgb(red, green, blue)`, each part from 0 to 255. Coordinates start at the top left corner, `x` to the right and `y` downwards. Everything is clipped to the canvas, so drawing off the edge is harmless.

A chart, because charts are useful

Nothing here is special to charts—it is arithmetic and rectangles.

```
use std.io
use std.draw
use std.math

fn chart(values: [Num], file: Text) -> Bool {
    let w = 420
    let h = 220
    let canvas = draw.canvas(w, h)
    draw.clear(canvas, draw.rgb(11, 11, 13))

    let top = values.max() ?? 1.0
    let gap = 10
    let bar = math.floor((w - gap) / values.len()) - gap

    for i in 0..values.len() {
        let tall = math.round(values[i] / top * (h - 50))
        draw.rect(canvas, gap + i * (bar + gap), h - tall - 24, bar, tall,
```

```

        draw.rgb(255, 161, 54))
    }

    draw.line(canvas, 0, h - 24, w, h - 24, draw.rgb(80, 76, 70))
    draw.text(canvas, 12, 16, "peak {top}", draw.rgb(180, 176, 170))
    give draw.save_png(canvas, file)
}

fn main() {
    io.say("chart written: {chart([3.0, 8.0, 5.5, 9.0, 2.0, 6.5], "chart.png")}")
}

```

Pixel by pixel

`draw.pixel` is the lowest level there is, and it is fast enough for real work: a 256 by 256 gradient is sixty five thousand calls and takes a few milliseconds.

```

use std.io
use std.draw
use std.math

fn main() {
    let size = 160
    let canvas = draw.canvas(size, size)

    var y = 0
    while y < size {
        var x = 0
        while x < size {
            let r = math.floor(x * 255 / size)
            let g = math.floor((x + y) * 128 / size)
            let b = math.floor(y * 255 / size)
            draw.pixel(canvas, x, y, draw.rgb(r, g, b))
            x = x + 1
        }
        y = y + 1
    }

    draw.save_png(canvas, "gradient.png")
    io.say("gradient {draw.size(canvas)} done")
}

```

The logo is a SPRFST program

`assets/logo/make_icons.spf` in this repository draws the SPRFST mark with nothing but `std.draw`, then writes it out at every size the app bundle needs:

```

sprfst run assets/logo/make_icons.spf

```

The icons you see in Studio and in the Dock came out of that file. If you want to change the mark, change the program.

Exercise

Draw a clock face: a circle, twelve ticks around the edge worked out with `math.sin` and `math.cos`, and two lines for the hands at the time `time.parts(time.now())` gives you.